

1. Objective

The objective of this lab is to clean and preprocess raw YouTube text data so that it can be reliably used for subsequent text mining and data mining tasks. Raw YouTube data collected from comments and captions is inherently noisy and unstructured, containing usernames, timestamps, interface-related text, emojis, punctuation, and inconsistent formatting.

This lab focuses on transforming such noisy textual data into a structured and normalized format through profiling, parsing, and preprocessing. The cleaned datasets produced in this lab are intended for direct reuse in future labs involving frequent pattern mining, clustering, and text mining techniques.

2. Tools and Libraries Used

The following tools and libraries were used to complete this lab:

- **Python 3.10+** – Primary programming language
- **pandas** – Creation and manipulation of DataFrames and CSV export
- **NLTK (Natural Language Toolkit)** – Tokenization, stopword removal, and lemmatization
- **webvtt-py** – Reading and extracting subtitle text from WebVTT files
- **re (Regular Expressions)** – Pattern matching for timestamps and text normalization
- **os** – File path and directory management

3. Dataset Description

3.1 comments.txt

The comments.txt file contains raw YouTube comments copied directly from a video. Manual inspection revealed the following structure and irregularities:

- Comments are organized in repeating blocks consisting of:
 - A username line (for example, @claytonharting9899)
 - A timestamp line (for example, 1 year ago)
 - One or more lines of comment text
- Interface-related text such as **“Reply”**
- Standalone numeric values representing like counts
- Extra blank lines between comment blocks
- Emojis and informal language within comment text

- Occasional non-English characters

Due to these inconsistencies, the file could not be processed directly and required custom parsing logic.

3.2 captions.vtt

The captions.vtt file contains subtitle data in WebVTT format. Initial inspection identified the following characteristics:

- A WEBVTT header and metadata lines
- Timestamp ranges using the --> format
- Caption text fragmented across multiple lines
- Inline caption tags such as <c> and embedded word-level timestamps
- Repeated caption lines caused by alignment and formatting differences

These issues made it necessary to remove timestamps, numeric lines, headers, and inline tags before meaningful text analysis could be performed.

4. Methodology / Procedure

4.1 Profiling the Data

Both comments.txt and captions.vtt were manually opened and inspected using a text editor prior to writing any code. This inspection helped identify structural patterns, formatting inconsistencies, and noise specific to each file.

For comments.txt, a consistent username → timestamp → text pattern was observed, along with interface-related noise and blank lines.

For captions.vtt, WebVTT headers, timestamp ranges, inline tags, and fragmented caption text were identified.

These observations directly guided the design of the parsing and cleaning pipeline.

4.2 Parsing Comments and Captions

Parsing Comments

Comments were parsed using regular expressions to detect timestamp lines such as “1 year ago” or “5 hours ago”. For each detected timestamp:

- The preceding line was treated as the username
- The timestamp line was stored separately
- All following lines up to the next timestamp were combined as the comment text
- Interface-related text such as “Reply” and numeric-only lines were ignored

The parsed data was stored in a pandas DataFrame with the following columns:

- username
- timestamp_text
- comment_text

Parsing Captions

Captions were initially read using the webvtt-py library. Due to inline formatting and word-level timestamps, additional preprocessing was required:

- Removal of <c> tags and embedded timestamps
- Deduplication of repeated caption lines
- Reconstruction of sentence-like segments using punctuation and word-count heuristics

The processed captions were stored in a DataFrame with the column:

- caption_sentence

4.3 Cleaning and Preprocessing Pipeline

The same cleaning and preprocessing steps were applied to both YouTube comments and caption sentences. This pipeline removes noise from the text and converts it into a clean set of meaningful words that can be used for text mining.

Normalization

Normalization is used to make all text consistent before further processing. In this step, the text is converted to lowercase and unnecessary elements such as URLs, usernames, punctuation, numbers, emojis, and extra spaces are removed. This helps reduce variation in the data and improves the quality of later analysis.

```
text = text.lower()

text = re.sub(r"http\S+", "", text)

text = re.sub(r"@w+", "", text)

text = re.sub(r"^[a-z\s]", "", text)

text = re.sub(r"\s+", " ", text).strip()
```

Tokenization

Tokenization splits the cleaned text into individual words. This step is important because most text mining algorithms work on words rather than full sentences.

```
tokens = word_tokenize(text)
```

Stopword Removal

Stopword removal removes very common words such as “the”, “is”, and “and” that do not carry meaningful information. Very short words are also removed to reduce noise in the dataset.

```
tokens = [w for w in tokens if w not in stop_words and len(w) > 2]
```

Lemmatization

Lemmatization converts words into their base or dictionary form. This helps group similar words together, such as “studying” and “studies” becoming “study”, which improves consistency in the cleaned data.

```
tokens = [lemmatizer.lemmatize(w) for w in tokens]
```

4.4 Exporting Cleaned Data

After preprocessing, rows that resulted in empty token lists were removed. The cleaned and structured datasets were then saved as CSV files so they can be reused directly in future labs without repeating the cleaning process.

The exported files are:

- cleaned_comments.csv
- cleaned_captions.csv

5. Code Implementation

This section summarizes the main components of the Python implementation used in this lab. The code is organized into separate parts to improve readability and maintainability.

5.1 Parsing Comments

This function reads the raw comments.txt file and extracts structured information such as username, timestamp, and comment text. It also removes interface-related text and combines multi-line comments into a single entry.

```
def parse_comments(filepath):  
    # Extracts username, timestamp, and multi-line comment text
```

5.2 Parsing Captions

This function processes the captions.vtt file. It removes timestamps, inline tags, and repeated caption fragments, then reconstructs the text into sentence-like units suitable for analysis.

```
def parse_captions(filepath):  
    # Handles WebVTT parsing, tag removal, deduplication, and sentence reconstruction
```

5.3 Cleaning Functions

This function applies the full text cleaning pipeline, including normalization, tokenization, stopwords removal, and lemmatization. It ensures that all text data is cleaned in a consistent way.

```
def clean_text_pipeline(text):  
    # Normalizes, tokenizes, removes stopwords, and lemmatizes text
```

5.4 Combined Cleaning Pipeline

The cleaning pipeline is applied to both comments and captions. The resulting cleaned tokens are stored in a new column called cleaned_tokens for each dataset.

```
comments_df["cleaned_tokens"] = comments_df["comment_text"].apply(clean_text_pipeline)  
captions_df["cleaned_tokens"] = captions_df["caption_sentence"].apply(clean_text_pipeline)
```

6. Results and Discussion

6.1 Results Summary

The table below summarizes the number of valid records obtained after parsing and cleaning the datasets.

Dataset	Parsed Records	Description
Comments	739	Valid comment blocks with cleaned tokens
Captions	1	Reconstructed caption sentence after deduplication

These results show that most of the useful information from the raw files was successfully extracted and cleaned.

6.2 Sample Cleaned Tokens

The table below shows an example of how raw text is transformed into cleaned tokens.

Raw Text (Excerpt)	Cleaned Tokens
"When Im feeling lonely, I look at the sky..."	[feel, lonely, look, sky, pick, direction, wave]

This example demonstrates how unnecessary words and symbols are removed while keeping the main meaning of the sentence.

6.3 Discussion

Compared to the raw data, the cleaned datasets are much more consistent and easier to analyze. The original comments and captions contained timestamps, emojis, interface text, and formatting issues that made direct analysis difficult.

The main challenge in this lab was handling inconsistent formatting in both the comment blocks and the WebVTT caption file. These issues were addressed through careful data profiling and flexible parsing logic. Overall, the preprocessing steps greatly improved data quality and prepared the datasets for future text mining tasks.

7. Conclusion

This lab highlights the importance of data cleaning and preprocessing in text mining. Raw YouTube data contains many types of noise that can negatively affect analysis if not handled properly. By profiling the data, converting it into structured formats, and applying normalization, tokenization, stopword removal, and lemmatization, the text data was transformed into clean and usable forms. These cleaned datasets will be used directly in future labs for pattern mining and text analysis.

8. References

- [NLTK Documentation](#)
- [Pandas Documentation](#)
- [Python re Module Documentation](#)
- [webvtt-py Documentation](#)